Python and Git 101

Python Programming and Data Analysis · Day 1 · Dataminded Academy

dataminded

Welcome to Day 1

Your trainers

- **Jan Vanbuel**, Dataminded
- **Oliver Willekens**, Dataminded, assisting in the room

Today

- Development environment
- Programming fundamentals
- Version control with Git

Ground rules

- Ask early, ask often
- Stuck for 5 minutes? Wave at us
- No programming experience assumed



Why learn this if AI can write the code?

Ask

You have to ask in the specialist's **vocabulary**

Judge

You have to read what comes back and decide if it is **right**

Choose

There are several approaches, and someone must own the **tradeoff**



Copilot today: how to use it

- Use it freely, in every exercise
- Ask it to explain errors and unfamiliar code
- Ask for a second approach, then compare the two
- Keep only lines you can explain to your neighbour
- You are the author of what you commit, Copilot is not



Judging code: three questions, in this order

1. **CORRECT**

   Does it do what it is supposed to do?

2. **READABLE**

   Will a colleague, or you in two months, understand it?

3. **PERFORMANT**

   Is it fast enough? Ask this last.



What you can do by 17:00

- Set up a Python environment and run a script from PowerShell
- Create a virtual environment per project and install packages with uv
- Write a script with functions, conditionals and loops, import a package into it, and run it from VS Code
- Read a stack trace and find the error it points to
- Automate a repetitive task with a script that reads and writes files
- Clone an Azure DevOps repository, commit on a branch, push and pull



Agenda

01	09:00 Setup check
02	09:30 Scripts, notebooks and running Python
03	10:15 Language fundamentals, lunch at 12:30
04	13:15 Standard library, files and exceptions
05	14:20 Projects with uv
06	15:05 Version control and Git
07	16:50 Wrap-up



Setup check: 30 minutes

```
uv --version
uv python list --only-installed
code --version
git --version
```

Something fails?

Do not debug alone. Call a trainer, setup problems are normal on managed laptops.

Scripts and notebooks



A script is a text file that runs top to bottom

```
# bench_summary.py
pressure_bar = 7.5

power_kw = 37.5
print(f"Power: {power_kw} kW at {pressure_bar} bar")

print("Done")
```

```
Power: 37.5 kW at 7.5 bar
Done
```

Scripts versus notebooks

	Script (.py)	Notebook (.ipynb)
Runs	Start to finish, every time, on demand or scheduled	Cell by cell, in any order you click
Great for	Automation, reusable tools, anything others run	Exploring data, plotting, explaining results
Weak at	Slow feedback when exploring	Repeatability and testing
In Git	Clean text, easy to compare versions	JSON with embedded output, noisy differences



The notebook trap: hidden state

```
# Cell 1                    # Cell 2                    # Cell 3
limit_bar = 9.0             limit_bar = 6.0             print(limit_bar)
```

- You ran cell 1, then cell 2, then edited cell 2 away and forgot about it
- Cell 3 prints a value that no longer appears anywhere in the visible code
- A colleague runs the notebook top to bottom and gets a different answer

Words you can now use script · notebook · cell · kernel · hidden state · reproducible

Python on your laptop



Where Python comes from

- Created by Guido van Rossum, first released in 1991
- Named after Monty Python, not the snake
- Open source, steered by the Python Software Foundation and a steering council
- On your laptop it is **CPython** 3.13, the reference interpreter, written in C



Which Python am I running?

```
Get-Command python
```

```
CommandType  Name        Source
-----------  ----        ------
Application  python.exe  C:\Users\you\AppData\Local\Microsoft\WindowsApps\python.exe
```

Watch out

A `WindowsApps` path is the Microsoft Store shortcut, not a real Python. It opens the Store or does nothing useful.

Run a script from PowerShell

```python
# hello.py
print("Compressor test bench online")
```

```
cd C:\Users\you\training
uv run python hello.py
```

```
Compressor test bench online
```

Running from VS Code

1

OPEN THE FOLDER

File, Open Folder, choose your project folder

2

SELECT THE INTERPRETER

Command Palette, Python: Select Interpreter, pick the one in `.venv` or the uv Python 3.13

3

RUN

Play button at the top right, or `uv run python hello.py` in the integrated terminal



Exercise 01: First script

`exercises/python/01-first-script`

Words you can now use interpreter · CPython · terminal · PATH · working folder · uv run

Language fundamentals



Variables are names for values

```
pressure_bar = 7.5
temperature_c = 82.3
run_id = "TB-014"

pressure_bar = 8.0
print(pressure_bar)
print(temperature_c)
```

```
8.0
82.3
```




The basic data types

```
run_id = "TB-014"        # str: text
cycles = 12              # int: whole number
flow_l_s = 18.4          # float: decimal number
is_stable = True         # bool: True or False
operator = None          # None: no value (yet)

print(type(flow_l_s))
```

```
<class 'float'>
```

Converting between types

```
text = "7.5"
pressure_bar = float(text)
print(pressure_bar + 1)        # 8.5

count = 3
print("Cycles: " + count)
```

```
TypeError: can only concatenate str (not "int") to str
```




Strings and f-strings

```
run_id = "TB-014"
pressure_bar = 7.5
print(f"Run {run_id}: {pressure_bar:.1f} bar")
print(f"{pressure_bar * 14.5038:.0f} psi")
print("TB" in run_id, run_id.lower())
```

```
Run TB-014: 7.5 bar
109 psi
True tb-014
```

Names are signposts

Written fast

```
p = 7.5
t = 82
calc(p, t)
```

Read easily

```
pressure_bar = 7.5
temperature_c = 82
to_kelvin(temperature_c)
```

Put the unit in the name. Constants in capitals: `ALARM_LIMIT_BAR = 8.5`

Exercise 02: Variables and types

`exercises/python/02-variables-and-types`

Words you can now use variable · assignment · type · string · float · f-string · convert

Lists: an ordered collection

```
pressures_bar = [6.8, 7.1, 7.5, 7.4, 7.9]
print(pressures_bar[0])
print(len(pressures_bar))
pressures_bar.append(8.2)
print(max(pressures_bar))
```

```
6.8
5
8.2
```




Indexing starts at zero

index	0	1	2	3	4
value	6.8	7.1	7.5	7.4	7.9
negative	-5	-4	-3	-2	-1

```
p = [6.8, 7.1, 7.5, 7.4, 7.9]
print(p[1:3], p[:2], p[-2:])
```

```
[7.1, 7.5] [6.8, 7.1] [7.4, 7.9]
```

Dictionaries: key to value

```
reading = {"run": "TB-014", "pressure_bar": 7.5}
print(reading["pressure_bar"])
reading["flow_l_s"] = 18.4
print(reading.get("power_kw"))
print(reading)
```

```
7.5
None
{'run': 'TB-014', 'pressure_bar': 7.5, 'flow_l_s': 18.4}
```




A choice: list or dict?

	list	dict
Find by	Position, in order	Name, the key
Fits	A series of the same kind of value	One record with different fields
Example	`pressures_bar[2]`	`reading["pressure_bar"]`

Copilot will pick one for you unless you say which you want.



Exercise 03: Collections

`exercises/python/03-collections`

Words you can now use list · index · slice · dict · key · value



Decisions with if, elif, else

```
pressure_bar = 9.4
if pressure_bar > 9.0:
    print("ALARM: over limit")
elif pressure_bar < 5.0:
    print("Warning: low pressure")
else:
    print("OK")
```

```
ALARM: over limit
```

Repeating with for

```
pressures_bar = [6.8, 7.1, 9.4]
for p in pressures_bar:
    print(f"{p:.1f} bar")

for i in range(3):
    print(i)
```

```
6.8 bar
7.1 bar
9.4 bar
0
1
2
```




Loop pattern: filter and summarise

```
pressures_bar = [6.8, 7.1, 9.4, 7.4, 9.1]
high = []
total = 0.0

for p in pressures_bar:
    total += p
    if p > 9.0:
        high.append(p)

print(high, round(total / len(pressures_bar), 2))
```

```
[9.4, 9.1] 7.96
```




Exercise 04: Control flow

`exercises/python/04-control-flow`

Words you can now use condition · branch · loop · iterate · block · indentation



Functions: name a piece of logic

```
def to_psi(pressure_bar):
    return round(pressure_bar * 14.5038, 1)

print(to_psi(7.5))
print(to_psi(10))
```

```
108.8
145.0
```

`return` is not print

```
def double_and_print(x):
    print(x * 2)

result = double_and_print(4)
print(result)
```

```
8
None
```

A function without `return` gives back `None`. Use `print` to show, `return` to pass on.

A small program made of functions

```
def mean(values):
    """Return the arithmetic mean of a list of numbers."""
    return sum(values) / len(values)

def summarise(values):
    return {"min": min(values), "max": max(values), "mean": mean(values)}

def main():
    pressures_bar = [6.8, 7.1, 9.4, 7.4]
    print(summarise(pressures_bar))

if __name__ == "__main__":
    main()
```




Type hints: reading a signature

```python
def to_psi(pressure_bar: float, decimals: int = 1) -> float:
    return round(pressure_bar * 14.5038, decimals)
```

- `pressure_bar: float` the parameter and the type it expects
- `decimals: int = 1` a type and a default value
- `-> float` the type of the return value

Hints are documentation that the editor and tools can check. Python does not enforce them when the script runs.



A choice: script, function or module?

	Use it when	The cost
Lines in the script	You do it once, in a few lines	Copy and paste when you need it again
A function	You repeat it, or it deserves a name	A little structure to read through
A module	Several scripts need the same functions	One more file, and an `import`

Copilot will pick one for you unless you say which you want.



A test says what correct means

```python
import pytest
import conversions

def test_bar_to_psi():
    assert conversions.bar_to_psi(1) == pytest.approx(14.5038)
```

Red: run, read the failure → Green: make it pass → Test first: write the next test

This loop is **test-driven development**.

Exercise 05: Functions

`exercises/python/05-functions`

Words you can now use function · argument · return value · type hint · test · assertion · test-driven development



The standard library



Batteries included: import and math

```
import math

diameter_m = 0.05
area_m2 = math.pi * (diameter_m / 2) ** 2
flow_m3_s = 18.4 / 1000
print(round(flow_m3_s / area_m2, 2))

print(math.isclose(0.1 + 0.2, 0.3))
```

```
9.37
True
```




pathlib: paths as objects

```python
from pathlib import Path

data_dir = Path("data")
log_file = data_dir / "run_014.csv"

print(log_file.name, log_file.suffix, log_file.stem)
print(log_file.exists())

for f in sorted(data_dir.glob("*.csv")):
    print(f)
```

```
run_014.csv .csv run_014
True
data\run_014.csv
data\run_015.csv
```




csv: reading rows

```
t_s,pressure_bar,temperature_c
0,6.8,71.2
10,7.1,74.5
20,7.5,78.9
```

```
import csv
with open("data/run_014.csv", newline="", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))
print(rows[0])
print(float(rows[1]["pressure_bar"]) + 1)
```

```
{'t_s': '0', 'pressure_bar': '6.8', 'temperature_c': '71.2'}
8.1
```




json: settings and nested data

```
import json
config = {"limit_bar": 9.0, "channels": ["pressure", "flow"], "operator": None}
text = json.dumps(config)
print(text)
back = json.loads(text)
print(back["channels"][0], back["operator"])
```

```
{"limit_bar": 9.0, "channels": ["pressure", "flow"], "operator": null}
pressure None
```

Exercise 06: Standard library

`exercises/python/06-standard-library`

Words you can now use standard library · module · import · path · glob · CSV · JSON



Files and exceptions



Reading a text file

```
from pathlib import Path

with open("data/run_014.csv", encoding="utf-8") as f:
    for line in f:
        print(line.strip())

text = Path("data/run_014.csv").read_text(encoding="utf-8")
```

```
t_s,pressure_bar,temperature_c
0,6.8,71.2
10,7.1,74.5
20,7.5,78.9
```

Writing and appending

```
lines = ["run,max_pressure_bar", "TB-014,7.9"]
with open("summary.txt", "w", encoding="utf-8") as f:
    for line in lines:
        f.write(line + "\n")

with open("summary.txt", "a", encoding="utf-8") as f:
    f.write("TB-015,8.2\n")
```

Mode `"w"` replaces an existing file without asking. `"a"` adds at the end, `"r"` only reads.

A traceback is a map to the error

```
Traceback (most recent call last):
  File "convert.py", line 2, in <module>
    pressure_bar = float(text)
ValueError: could not convert string to float: 'seven'
```

- Line 1 is only a header, it says: read on
- The middle shows the file, the line number and the code that failed
- The last line names the error type and what went wrong. Read it first.



Handling errors with try / except

```
text = "seven"
try:
    pressure_bar = float(text)
except ValueError:
    pressure_bar = None
    print(f"Skipping bad value: {text!r}")
print(pressure_bar)
```

```
Skipping bad value: 'seven'
None
```




Exercise 07: Files and exceptions

`exercises/python/07-files-and-exceptions`

Words you can now use file handle · encoding · exception · traceback · raise · catch



Projects with uv



uv and pip: the same job, fewer steps

	pip workflow	uv workflow
Environment	`python -m venv .venv`	`uv init`, then automatic
Activate	`.venv\Scripts\Activate.ps1`	Not needed: `uv run`
Install	`pip install matplotlib`	`uv add matplotlib`
Declare	`requirements.txt`, by hand	`pyproject.toml`, updated by uv
Pin versions	`pip freeze`, if you remember	`uv.lock`, automatic



Create a project and add a package

```
uv init bench-tools
cd bench-tools
uv add matplotlib
uv run python main.py
```

```
Creating virtual environment at: .venv
Resolved 12 packages in 330ms
Installed 11 packages in 68ms
 + matplotlib==3.11.2
 + numpy==2.5.4
 ...
```




pyproject.toml declares, uv.lock pins

```toml
[project]
name = "bench-tools"
requires-python = ">=3.13"
dependencies = [
    "matplotlib>=3.11.2",
]
```

Commit

`pyproject.toml`, `uv.lock`, `.python-version` and your `.py` files

Do not commit

`.venv/`, big and machine specific



Exercise 08: A uv project

`exercises/python/08-uv-project`

Words you can now use virtual environment · package · dependency · lockfile · pin · sync



Python goes further: not today

- **Styles**: procedural (today), object-oriented with classes and dataclasses, functional
- **Language**: decorators, generators and comprehensions, context managers
- **Debugging**: the VS Code debugger, with breakpoints and stepping, instead of `print`
- **Scripts that grow**: `logging` instead of `print`, arguments with `argparse`

- **Performance**: measure first (`timeit`, a profiler), then NumPy
- **Concurrency**: threads, processes, `asyncio`
- **Extending**: C, or Rust through PyO3
- **Packaging**: your own code, installable by colleagues



Version control



The final_v3 problem

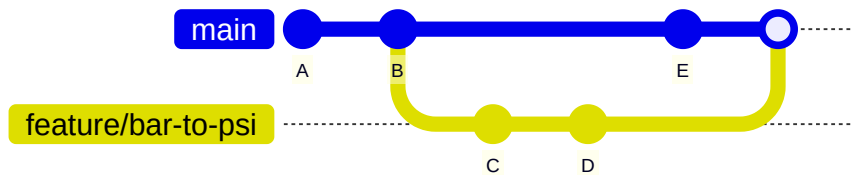
```
\\fileserver\compressors\testbench\
    analyse_run.py
    analyse_run_final.py
    analyse_run_final_v3_USE_THIS_ONE.py
```

- **History**: which version produced last week's report?
- **Who and why**: who changed the conversion factor?
- **Undo**: how do I get back to the version that worked?
- **Collaborate**: two people edit at once, whose version wins?
- **With AI**: `git diff` shows exactly what an assistant changed, so you keep what is good and undo the rest

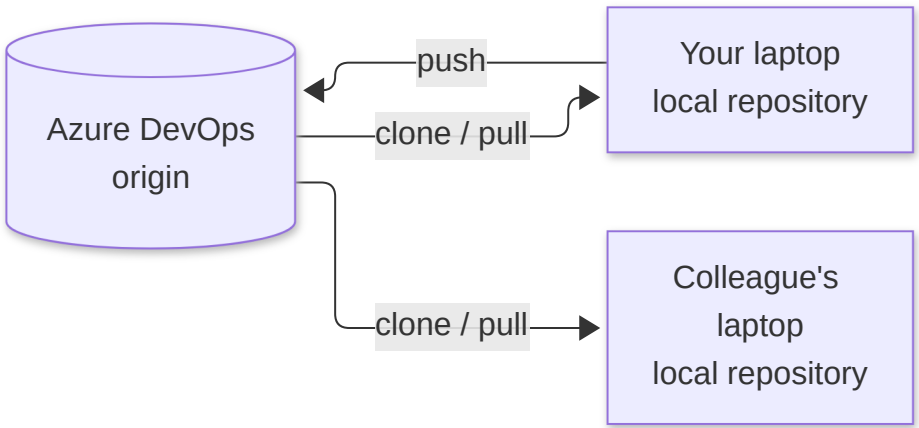


Repository, commit, branch

- **Repository**: a project folder plus its full history
- **Commit**: a saved snapshot with a message, author and date
- **Branch**: a movable label on a commit
- Work on a branch does not touch `main` until you merge it

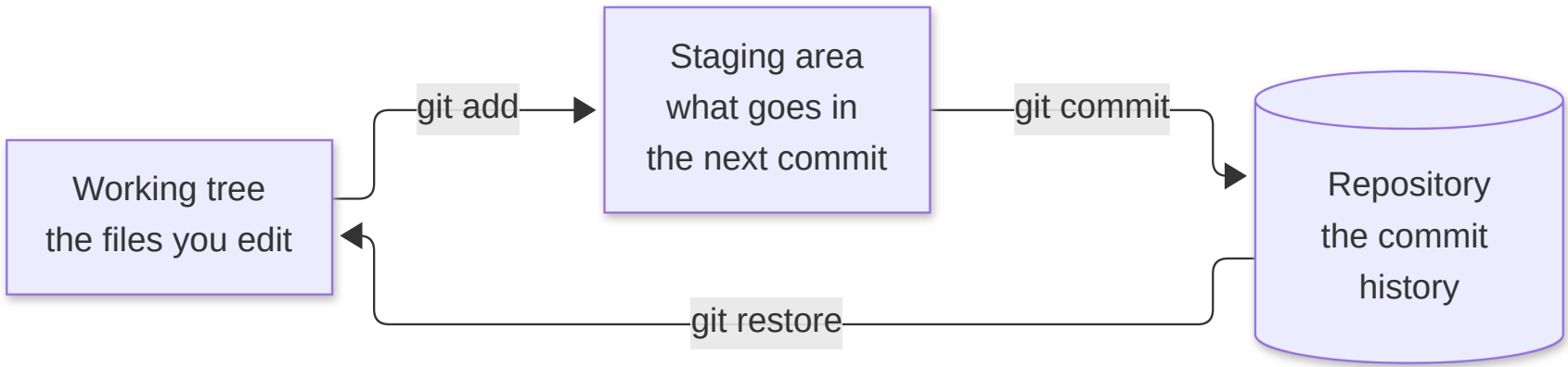


Git, Azure DevOps and the remote



- **Git** is the free tool on your laptop, **Azure DevOps** hosts the shared copy
- A **remote** is that shared copy, by convention called `origin`

Three areas on your laptop



- You choose what goes into a commit by **staging** it first
- `git status` tells you which area each change is in

Words you can now use repository · commit · branch · remote · working tree · staging area

Git on the command line



Set up: identity and a first repository

```
git config --global user.name "Your Name"
git config --global user.email "you@atlascopco.com"
mkdir testbench
cd testbench
git init -b main
git status
```

- Identity: once per laptop, same email as your Azure DevOps account
- `git status` never changes anything: run it all the time



Stage and commit

```
git add conversions.py
git commit -m "Add psi to bar conversion"
```

```
[main (root-commit) a055887] Add psi to bar conversion
 1 file changed, 2 insertions(+)
```

- Message: short, imperative, one logical change: "Fix psi to bar factor", not "update"
- `git add .` stages everything: check `git status` first



Read the history: log and diff

```
git log --oneline
```

```
9b221ad Add first measurement
a055887 Add psi to bar conversion
```

```
git diff            # edits not yet staged
git diff --staged   # what the next commit will contain
```

- If the output is long, Git opens a pager: **q** leaves it



Exercise time: first commit

`exercises/git/01-first-commit`

About 10 minutes: `uv run python exercises/git/setup_exercise.py 1`



Old versions: show and restore

```
git show a055887                    # one commit: message and diff
git show HEAD~1:conversions.py      # a file as it was one commit ago
git restore --source=HEAD~1 conversions.py
git restore conversions.py          # discard edits since last commit
```

- `HEAD` is the latest commit, `HEAD~1` the one before it
- `git restore <file>` throws your edits away: there is no undo for that



Exercise time: history

`exercises/git/02-history`

About 10 minutes: `uv run python exercises/git/setup_exercise.py 2`



Branch and merge

```
git switch -c feature/bar-to-psi
git add conversions.py
git commit -m "Add bar to psi conversion"
git switch main
git merge feature/bar-to-psi
```

```
Updating 9b221ad..1cd9279
Fast-forward
```

- **Fast-forward** if `main` did not move; otherwise Git makes a merge commit
- `git branch` lists branches, `git branch -d <name>` deletes one



When two branches edit the same lines

```
CONFLICT (content): Merge conflict in conversions.py

def psi_to_bar(psi):
<<<<<<< HEAD
    return psi * 0.068948
=======
    return psi * 0.06895
>>>>>>> feature/a
```

- A conflict is Git asking you to decide, nothing is lost
- Edit the file, delete the marker lines, then `git add` and `git commit`
- Changed your mind? `git merge --abort`



Exercise time: branches

`exercises/git/03-branches`

About 15 minutes: `uv run python exercises/git/setup_exercise.py 3`

Words you can now use stage · commit message · diff · HEAD · merge · fast-forward · conflict



Remotes and Azure DevOps



Clone a repository

```
git clone https://dev.azure.com/<organisation>/<project>/_git/<training-repo>
cd <training-repo>
git remote -v
```

```
origin  https://dev.azure.com/<organisation>/<project>/_git/<training-repo> (fetch)
```

- The first time, a browser window asks you to sign in (Git Credential Manager)
- `clone` creates the remote `origin`; the URL is on the **Clone** button in Azure DevOps



Push and pull

```
git push -u origin feature/bar-to-psi   # first push of a new branch
git push                                 # later pushes
git pull                                 # fetch and merge new commits
git fetch                                # only download, change nothing
```

- `-u` links your local branch to the remote one, needed only once
- `pull` is `fetch` plus merge: commit or restore your edits first

Exercise time: Azure DevOps remote

`exercises/git/04-azure-devops-remote`

About 15 minutes: clone, branch, commit, push



The pull request flow



- A pull request (PR) proposes merging your branch into `main`; colleagues read the diff first
- **Complete**: pick the merge type (squash keeps `main` readable), tick **Delete source branch**
- Afterwards update your laptop: `git switch main`, `git pull`

Exercise time: pull request

`exercises/git/05-pull-request`

About 20 minutes: PR into main, review, complete, pull



Keep files out: .gitignore

```
# .gitignore
.venv/
__pycache__/
data/raw/
```

- One pattern per line; commit the file itself so everyone shares it
- Never commit passwords, tokens or large raw data
- Day 2 uses it for data files and notebooks



Three ways to use Git

	Command line	VS Code	SourceTree
Advantages	Every feature, same everywhere, easy to copy into notes	Already open, diff and conflict views built in	Clear graph of branches, point and click
Disadvantages	Commands to learn, easy to mistype	Hides the commands, fewer rare features	Extra install, less common on managed laptops
Use it when	You need full power, or help from a colleague or a doc	Daily edit, commit, push work next to the code	You want to browse a complex history visually



Demo: Git in VS Code

Optional practice: `exercises/git/06-git-in-vscode`

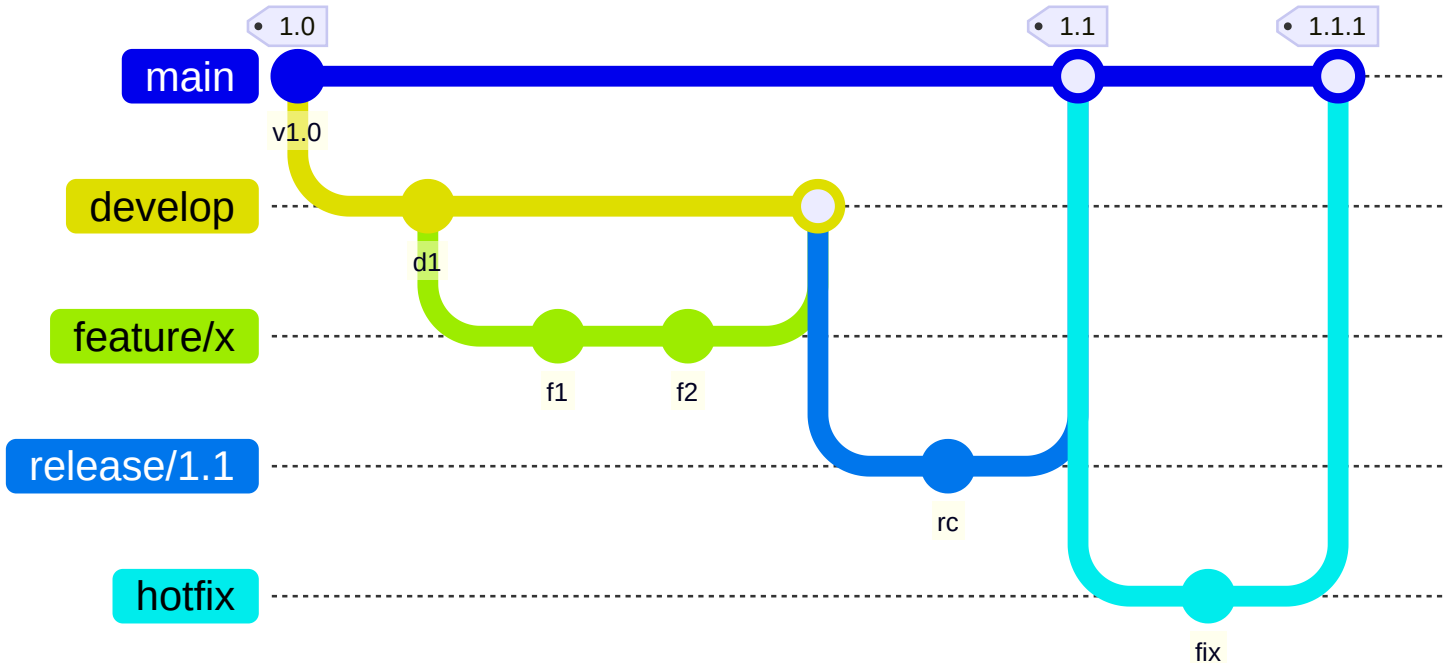
Words you can now use clone · origin · push · pull · fetch · pull request · review



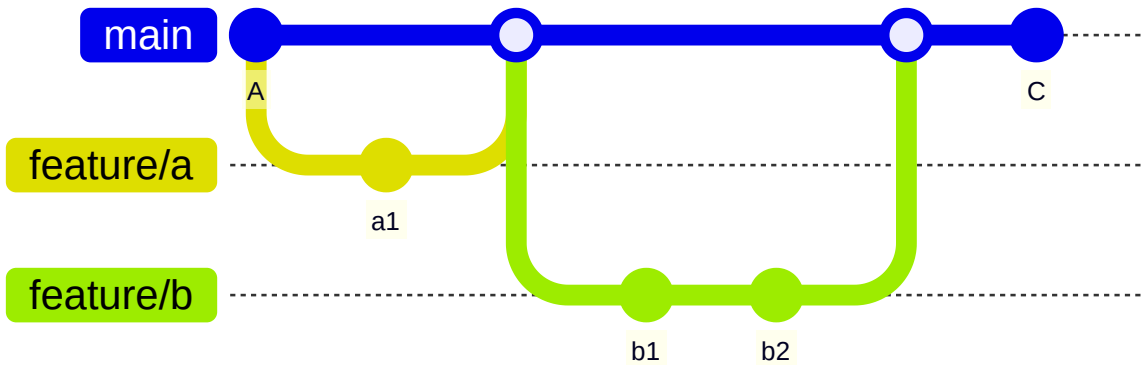
Branching models



GitFlow


1.0
1.1
1.1.1
main
v1.0
develop
d1
feature/x
f1
f2
release/1.1
rc
hotfix
fix

Trunk-based development



- One long-lived branch, `main`; short branches merged through a pull request

Our recommendation

	GitFlow	Trunk-based
Fits	Versioned products, several releases in support	Continuous delivery, scripts, analyses, small teams
Cost	Many branches, large merges, more conflicts	Discipline: small changes, review, tests

- **Use trunk-based**: short-lived `feature/...` branches and PRs; day 3 adds a branch policy on `main`
- Exists, not today: `git rebase`, `git cherry-pick`, `git stash`

Words you can now use trunk-based · GitFlow · feature branch · short-lived · branch policy



What you can do now

Python

- Run a script from PowerShell and from VS Code
- A virtual environment per project with uv
- Functions, conditionals, loops, importing a package
- Read a stack trace
- Read a folder of files and write a summary

Git

- Clone an Azure DevOps repository
- Commit on a branch
- Push, merge through a PR and pull
- Read history, undo a change, recognise a conflict

Today: the **vocabulary** and the tradeoffs. Day 3: you use them to direct and review Copilot and coding agents.

Git cheat sheet

Local

```
git status
git add <file>
git commit -m "message"
git log --oneline
git diff
git restore <file>
git switch -c <branch>
git merge <branch>
```

Remote

```
git clone <url>
git switch main
git pull
git switch -c feature/x
git push -u origin feature/x
# open the PR, complete it, repeat
```




Before day 2, and beyond

Before day 2

- A working uv environment
- Redo the fundamentals you found hard
- One full clone, commit, push, pull cycle with a PR

Read more

- Pro Git book (`git-scm.com/book`)
- Learn Git Branching (`learngitbranching.js.org`)
- Python tutorial (`docs.python.org/3/tutorial`)
- uv docs (`docs.astral.sh/uv`)

Next days

- Day 2: pandas, NumPy, SciPy, Matplotlib, scikit-learn
- Day 3, going deeper: tests, linting, pipelines, coding agents and reviewing their work





Thank you

See you on day 2

